

Classification and Regression Trees, and, Random Forests

1 Forecasting Supreme Court Decisions

Model: Classification and Regression Trees (CARTs), Random Forests (RFs)

The Analytics Edge: Political scientists and legal academics constantly seek to understand what motivates the decision of justices. In 2002, a group of academics predicted the decisions of the U.S. Supreme Court using information on past decisions and compared them to the expert judgments of legal academics and professionals (see Martin et al. [2004]). Using an interpretable analytics technique known as Classification and Regression Trees, the model got 75.0% of the court's affirm/reverse results correct, while experts collectively got 59.1% right. The 83 experts were chosen from academics and attorneys and they were asked to predict outcomes in cases only in their expertise area. At the individual justice level, the model got 66.7% decisions correct while experts got 67.9%. Using analytics can provide an edge in a traditionally qualitative application. Their analysis showed that at the individual justice level, the experts got better predictions on the more "predictable" justice votes, while the model did a better job at the more "unpredictable" justice votes. Combining the two approaches, i.e, the consistent and unemotional nature of a model with the intuition and knowledge of experts, it provides a new fruitful approach in law.

1.1 Overview (Supreme Court of the United States)

The Supreme Court is the highest court in the federal judiciary of the United States. It consists of a chief justice and eight associate justices, who are nominated by the U.S. President and confirmed by the Senate. Once appointed, judges have lifetime tenure, unless they retire, resign, or are removed from office. Each justice has one vote. Because of the importance and nature of their decisions—many high-profile cases expose ideological beliefs—justices are often categorized as conservative, moderates, or liberal based on their votes in previous cases. The Supreme Court decisions impact a variety of social and economic questions. In 2002, two such important cases were:

1. *Grutter vs Bollinger*. This case dealt with the constitutionality of affirmative action (i.e., the policy of favouring members of a disadvantaged group who have suffered from discrimination). This case was upheld by the Supreme Court ruling that the affirmative action policy of the University of Michigan law school was justified. The case was affirmed 5–4.
2. *Lawrence vs Texas*. This case dealt with the right to engage in consensual homosexual sodomy. In a 6–3 ruling, the court reversed and struck down the sodomy law in Texas making same-sex sexual activity legal in the U.S.
3. *Roe vs Wade*. This case dealt with the right to have an abortion. In 1973, in a 7–2 ruling, the supreme court affirmed the US district court's ruling in favor of Jane Roe (pseudonym for Norma McCorvey) in her right to abortion plea. This ended many abortion laws in the US and led to an abortion debate across the country which still runs today. In 2022, the supreme court repealed the decision ending the constitutional right to abortion.

Many of the court decisions are hard to predict. Martin et al. [2004] predicted the 2002 term court decisions using a few simple predictor variables. The model they adopted is indifferent to many of the specific legal and factual aspects of the case that legal experts might use when making predictions. In particular, Martin et al. [2004] used data for the period 1994–2001 (one of the longest periods of time with the same justices) to predict 2002 decisions. The dataset contains the following predictors:

- The circuit of origin for the case: 1–11, FED, DC;
- The issue area of the case: Criminal procedure, Civil rights, First amendment, Economic activity, Federal taxation, ...;
- The type of petitioner: Business, City, Employee, American Indian, Politician, ...;
- The type of respondent: Business, City, Employee, American Indian, Politician, ...;
- The ideological direction of the lower court ruling: Liberal, or, Conservative; and,
- Whether or not the petitioner argued the constitutionality of a law or practice (0 (No) or 1 (Yes)).

The data also contains one response variable: **result**. It is 1 if the decision was *reversed* by the Supreme Court and 0 if it was *affirmed*.

Further details about the *The Supreme Court Forecasting Project* are available at <http://wusct.wustl.edu>.

Key Question: Is it possible to predict whether the Supreme Court judges will affirm or reverse the lower court decisions (at the individual judge level, and at the overall case level) using interpretable analytics?

In the study of Martin et al. [2004], they used 623 cases from 1994 to 2001 and predicted on the 68 cases for year 2002. They used a two step approach:

- Step 1: A tree was built to predict unanimous liberal decisions and a tree was used to predict unanimous conservative decisions. If the predictions from these two trees did not conflict, then the decision was made using the appropriate tree.
- Step 2: For the complex cases, where the decisions were not unanimous, trees were built for each judge and then the majority was used to predict decisions.

1.2 Summary

Data: The data contain information on the cases and corresponding decisions made by the U.S. Supreme Court during the period 1994–2001.

Model: CARTs and Random Forests can be used to forecast the decisions made by the Supreme Court. For each case, the decision is either affirm or reverse, so the model output is a binary variable. Here CART provides a very interpretable model to try and understand how decisions are made and to help with predictions. Random forests tend to typically have better accuracy but are less interpretable.

Value and Decision: The model predictions could be used in litigant’s decisions to press an appeal and in settlement negotiations. Specifically it provides a quantitative approach to aid with qualitative decisions.

2 Classification and Regression Trees (CART)

2.1 Introduction

Suppose you want to predict a value, or response, Y from predictors $X_1, \dots, X_p$. Models such as **linear regression** or **logistic regression** are global models in which a single predictive formula is supposed to hold over the entire data region (with linear dependence between dependent and independent variables). One might be able to use nonlinear regression techniques if the data interact in complex, nonlinear ways. However, these models are often less interpretable. An alternative approach is discussed next.

The idea is to divide (partition) the space of predictor variables into a number of simple regions, and then use simple rules - such as the mean (regression) or the mode (classification) of the observations - in the space partitioned to make predictions. By partitioning the sub-regions recursively (recursive partitioning), we can develop regions where the models within each region are very simple. Such models where the regions are defined using boxes (hyper-rectangles) have proved to be very useful in areas such as healthcare and law as we will see.

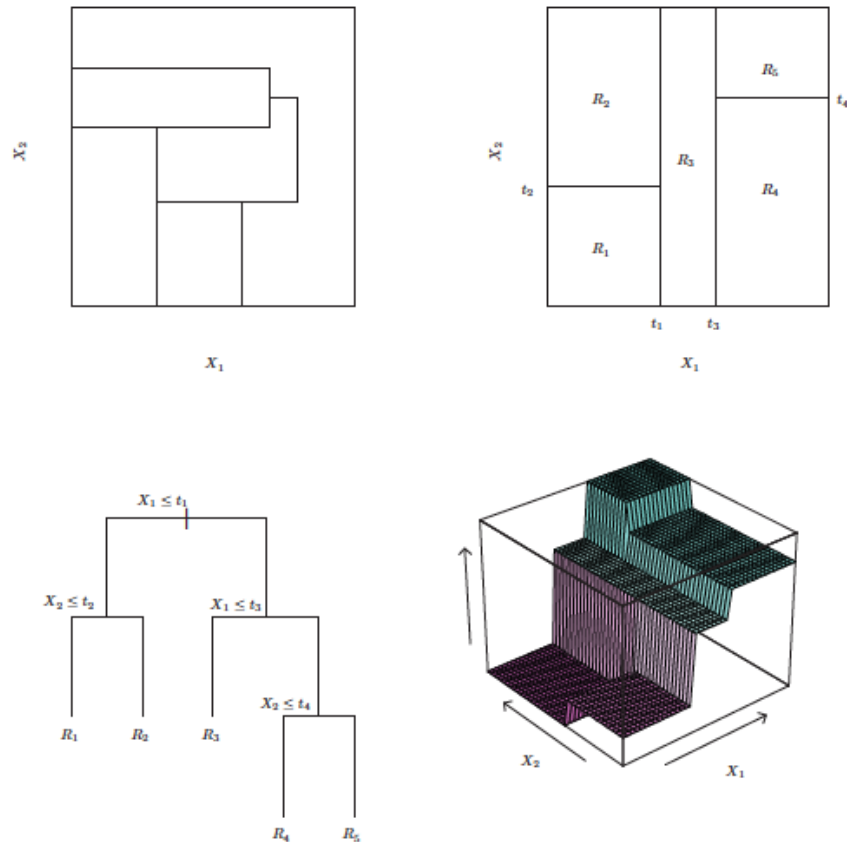


Figure 1.1: CART's recursive partitioning approach for a regression problem. Top Left: A partition of two-dimensional feature space that could not result from recursive binary splitting. Top Right: The output of recursive binary splitting on a two-dimensional example. Bottom Left: A tree corresponding to the partition in the top right panel. Bottom Right: A perspective plot of the prediction surface corresponding to that tree. *Courtesy: [James et al., 2014, Figure 8.3].*

We can read the model in the previous page as follows:

Check if $X_1 \leq t_1$:

- If yes, check if $X_2 \leq t_2$;
 - If yes, we are in region R_1 .
 - If no, we are in region R_2 .
- If no, check if $X_1 \leq t_3$:
 - If yes, we are in region R_3 .
 - If no, check if $X_2 \leq t_4$:
 - If yes, we are in region R_4 ;
 - If no, we are in region R_5 .

The example in Figure 1.2 illustrates a similar approach for a classification problem, that is, to predict $\times$ or $\circ$ based on the splits and the majority number in each split.

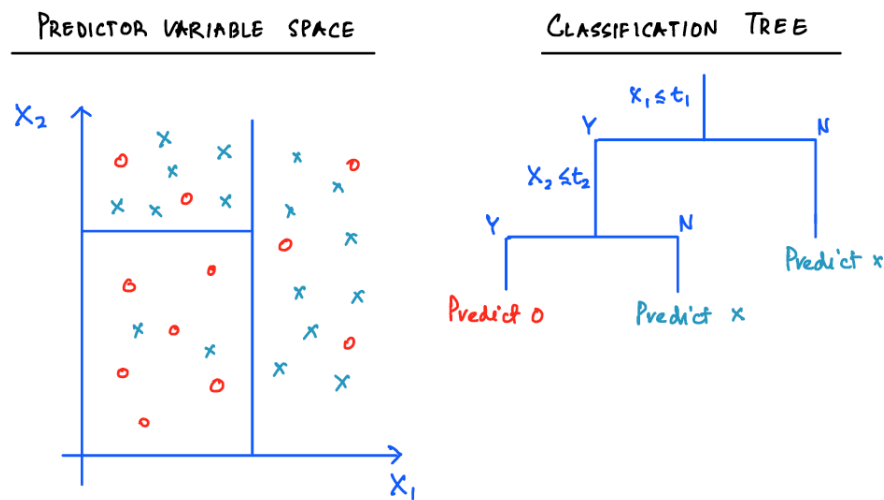


Figure 1.2: CART's recursive partitioning approach for a classification problem.

As we shall see, CARTs have two main advantages:

- The model is easily interpretable, even to non-experts.
- The model does not assume a linear relationship between input and output variable, and hence can capture nonlinearities in the data (see the illustration in Figure 1.3).

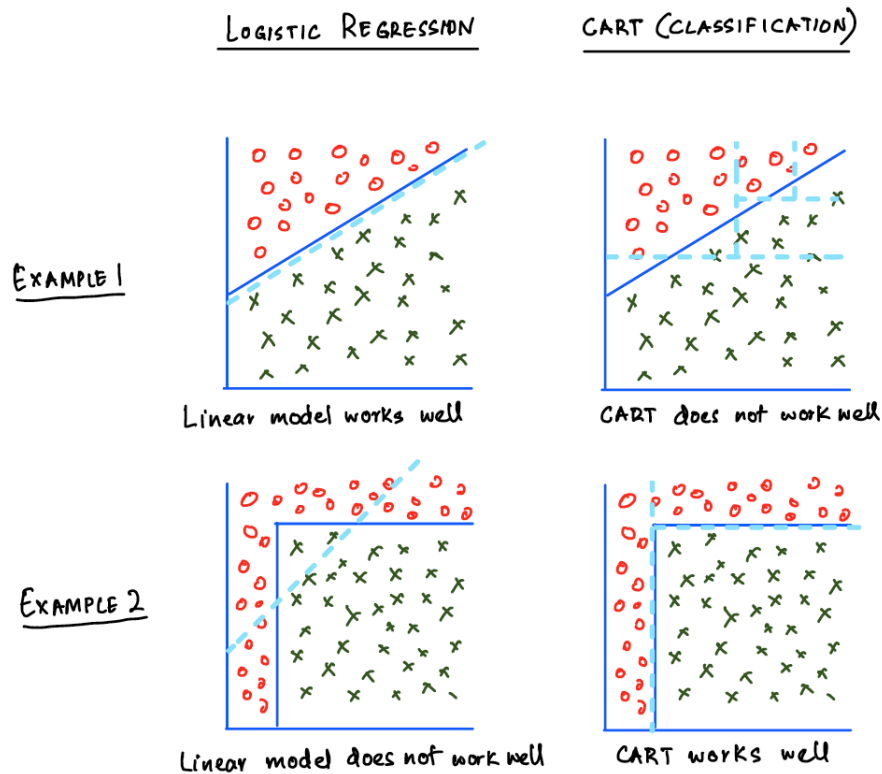


Figure 1.3: Comparison between linear model (logistic regression) and CART.

2.2 Finding the tree (regression)

To create a tree, we want to divide the space of predictor variables $x_1, \dots, x_p$ into M distinct and non-overlapping regions $R_1, \dots, R_M$. In CART, these regions are defined by boxes (hyper-rectangles), where the cuts are parallel to the axis. From an optimization standpoint, we can define the problem as follows:

$$\min_{R_1, \dots, R_M \in \mathcal{R}} \sum_{m=1}^M \sum_{i \in R_m} (y_i - \bar{y}_{R_m})^2,$$

where $R_1, \dots, R_M$ are the regions (possibly open boxes or open hyper-rectangles) partitioning the space with $R_i \cap R_j = \emptyset, \forall i \neq j$. We denote this feasible region by $\mathcal{R}$. Here y_i is the i -th observation in R_m , and $\bar{y}_{R_m}$ is the mean value of all observations in R_m . This is a difficult optimization problem to solve.

We are thus often forced to use local search procedures based on heuristics that create “good” trees in “reasonable” time. A well-known greedy strategy was introduced by Breiman [1984], a distinguished statistician at UC Berkeley. The strategy he proposed, which is the main building block of CARTs, works as follows:

- Start with all observations in a single region.
- Select the predictor variable x_j and cut point s such that splitting into region $\{(x_1, \dots, x_p) | x_j \geq s\}$ and $\{(x_1, \dots, x_p) | x_j < s\}$ results in the smallest residual sum of squares (RSS) (alternatively referred

to as sum of squared errors (SSE)):

$$\min_{j=1,\dots,p} \min_s \sum_{i:x_{ij} < s} (y_i - \bar{y}_{R_1})^2 + \sum_{i:x_{ij} \geq s} (y_i - \bar{y}_{R_2})^2$$

where $R_1 = \{(x_1, \dots, x_p) | x_j < s\}$ and $R_2 = \{(x_1, \dots, x_p) | x_j \geq s\}$. Once this problem is solved (by exhaustive search), we find j^* and s^* . We use this as the first split.

- Now, we repeat this step for each sub-region, individually and choose from the “best” split among them. We continue until a stopping criterion is met, such as not having too few observations in any region.

For example, after the first split - we will have two regions, after the second split - we will have three region and so on. Observe that in each step of the split, we are trying to maximize the improvement in going from the set $L \cup R$ (left union right) to partitioning it into L (left) and R (right):

$$\underbrace{\sum_{i \in L \cup R} (y_i - \bar{y}_{L \cup R})^2}_{\text{to maximize}} - \underbrace{\left(\sum_{i \in L} (y_i - \bar{y}_L)^2 + \sum_{i \in R} (y_i - \bar{y}_R)^2 \right)}_{\text{to minimize}}.$$

Note that greedy splits are made at each step without looking necessarily at the best split that might lead to a better tree in future steps (hence it is a locally optimal greedy strategy).

2.3 Finding the tree (classification)

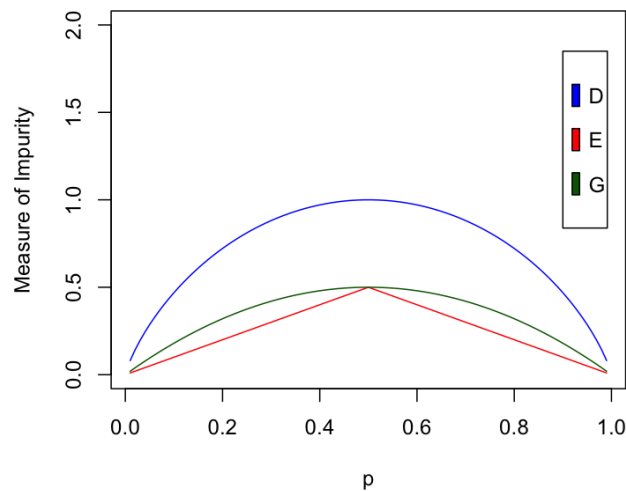


Figure 1.4: Comparison between Classification Error rate (E), Gini Index (G) and Entropy (D).

A classification tree is built in almost the same manner as a regression tree, except that the criterion used to select predictor variable and cut point (for each decision node) is a measure of impurity. A split is pure

if, after the split, for all branches, all the instances in the branch belong to the same class. Some of the most common measures of impurity are:

1. Classification error rate given by

$$E = 1 - \max_{k=1,\dots,K} p_{mk}$$

which assumes that the class with maximum proportion is to be predicted.

2. Gini Index given by

$$G = \sum_{k=1}^K p_{mk}(1 - p_{mk}) = 1 - \sum_{k=1}^K p_{mk}^2$$

where p_{mk} is the proportion of training observations in the m -th region from the k -th class. The last equality holds since $\sum_{k=1}^K p_{mk} = 1$.

3. Entropy (or Information Gain) given by

$$D = - \sum_{k=1}^K p_{mk} \cdot \log_2(p_{mk}).$$

The final two indices take a value near 0, as long as the p_{mk} values are either very close to 0 or 1 indicating almost all instances are in one class (pure) (see Figure 1.4). These measure help capture how impure a bucket is. The package `rpart` - the most common R package for CARTs - uses the Gini Index for splitting.

In some splits for CART models, you might end up with a situation like the one illustrated in Figure 1.5, where the bottom split seems superficial. However, note that such a split might give improved node purity: although the same response is predicted in both leaves ("x"), it might be that in one leaf the observations are all in the same class (so you are more sure) while in the other one there might be some impurity (so you are less sure).

Note that, in CART models, splitting on qualitative predictor variables is straightforward too.

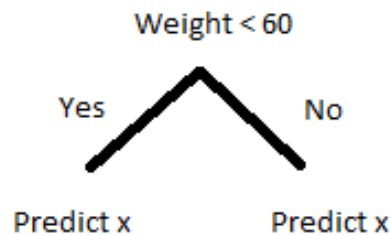


Figure 1.5: Splitting where the bottom split seems superficial.

2.4 Pruning a tree

There is a trade-off between the model interpretability and its performance on the training set:

- Small tree with fewer splits $\Rightarrow$ More interpretable (lower variance), but poorer fit in training set (higher bias);
- Larger tree with many splits $\Rightarrow$ Less interpretable (higher variance), better fit in training set (lower bias).

Yet, what often really matters is the model performance on the test set, meaning that we are interested in the bias-variance trade-off (illustrated in Figure 1.6)—where bias is the error introduced by approximating a function f with a simpler function $\hat{f}$, while variance refers to how the estimation would change if we used a different training set. This leads to the following trade-off:

- More complex model $\hat{f} \Rightarrow$ Low bias and high variance, since the model will capture many features of the data;
- Simpler model $\hat{f} \Rightarrow$ High bias and low variance.

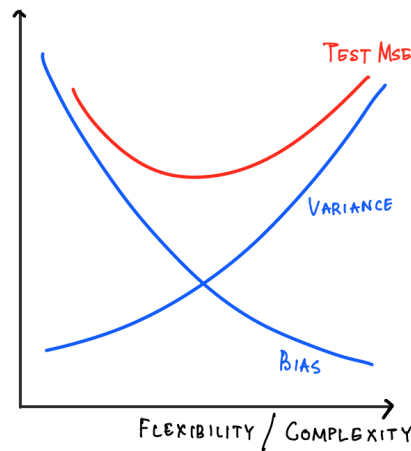


Figure 1.6: Bias-variance trade-off.

To control the bias-variance trade-off (or, simply, the model complexity), CARTs use *pruning*. We first grow a large tree and then prune it back to get a subtree with the objective of getting low test error rates. For a regression tree, pruning can be easily implemented by introducing a cost complexity function, which measures the trade-off between the fit to the training data and the model complexity. For any value of α , we want to find the sub-tree $T \subseteq T_0$ of the original tree (T_0) such that:

$$\min_{T \subseteq T_0} \sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \bar{y}_{R_m})^2 + \alpha |T|,$$

where $|T|$ is the number of leaves (terminal nodes) in the tree. When $\alpha = 0$, $T = T_0$ (original tree). As α increases, there is a price to be paid for having too many leaves and so we look for smaller sub-trees (similar to LASSO). To find the best value of α , one can use cross-validation methods. In doing pruning of trees for classification problems, `rpart` uses the proportion of misclassified instances and number of terminal nodes.

3 Random Forests

Random Forests is a popular technique to solve classification and regression problems and was introduced by Breiman [2001]. It builds on a combination of tree predictors (ensemble method) that operates by constructing a multitude of trees during the training phase and asking each tree to output the mode of the classes (most popular class in classification) or mean predictions (in regression). Then we pick the majority (classification) or mean (regression) across trees to make predictions. From a mathematical standpoint, we can interpret the function approximation of random forests as follows:

$$f(x) = \sum_{t=1}^B \frac{f_t(x)}{B},$$

where B is the number of trees in the ensemble, and each $f_t(x)$ is a CART that trains on a subset of the data that is chosen randomly with replacement (this is known as *bootstrapping*). Bagging (bootstrap aggregation) uses the above B bootstrap samples and averages the output (for regression) or takes majority (for classification). One of the challenges with this approach is that, if we use the same algorithm, then the predictor variables chosen will be highly correlated. This can be improved by learning trees on randomly chosen subsets of input variables. Note that RF provide less interpretability but often better predictive power than CART. Naturally, the larger the number of trees the larger the time needed to build the ensemble (a similar concept applies to the node sizes; smaller the node size, the larger the time needed to build the ensemble).

In a typical implementation, a random sample of $m \approx \sqrt{p}$ predictors (for classification) and $m \approx p/3$ predictors (for regression) are considered at each split of the tree and the split is made based only on one of this candidate variables. This helps decorrelate the trees chosen by the Random Forest method and reduces the variance.

(Rough) algorithm for Random Forests. For a selected number of trees B :

- Sample observations (with replacement) to create a subset of data;
- Build a tree. At each node:
 - For a chosen value of m , select only m random predictor variables from the entire set;
 - Find the best among m predictors;
 - At the next node choose another m random predictors and repeat.

In random forests, you can obtain an OOB (out of bag error) estimate by computing the errors for each training observation using the predictions from trees that did not use this observation in their bootstrap samples when they were being built and aggregating it. This serves a similar purpose as cross validation error.

4 Summary of output from CART in R

Package: `rpart`, `rpart.plot`

Function: `rpart`, `prp`

For classification tree, when you print the output - for each node, we see: node number, variable used for splitting, number of observations in the node, loss error in the node, predicted value, and predicted

probabilities. Note that the children of node $\mathbf{x}$ are numbered $2\mathbf{x}$ and $2\mathbf{x}+1$. Also, note that a variable might appear many times in the tree. The symbol $*$ indicates a terminal node.

5 Summary of output from Random Forest in R

Package: `randomForest`
 Function: `randomForest`

1. `model$forest` - stores the details of the entire forest.
2. `model$predicted` - the predicted values of the input data based on out-of-bag samples.
3. `model$confusion` - (classification only) the confusion matrix of the prediction (based on OOB data).
4. `model$votes` - (classification only) a matrix with one row for each input data point and one column for each class, giving the fraction or number of (OOB) ‘votes’ from the random forest.

References

- L. Breiman. *Classification and Regression Trees*. Chapman & Hall/CRC, 1984.
- L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning with applications in R*. Springer Texts in Statistics. Springer, New York, 2014.
- A. D. Martin, K. M. Quinn, T. W. Ruger, and P. T. Kim. Competing approaches to predicting supreme court decision making. *Perspectives on Politics*, 2(4):761–767, 2004.

Additional notes

In using CART for doing a classification task, in going from a node A to nodes A_L and A_R (partitioning A), the reduction in impurity is measured by:

$$\underbrace{\text{Impurity}(A) - \left(\underbrace{\frac{N(A_L)}{N(A)} \text{Impurity}(A_L) + \frac{N(A_R)}{N(A)} \text{Impurity}(A_R)}_{\text{to minimize}} \right)}_{\text{to maximize}}$$

where $N(A)$ is the number of observations in node A . The goal is to find a split of the node with largest reduction in impurity or smallest impurity after split.

Example 1. Let us look at an example where we want to split node A (or region A) which has 16 appearance of '1's and 9 appearance of '0's. We will compare the reduction in impurity for two different splits given in Figure 1.7. For this example we use Gini index as the measure of impurity.

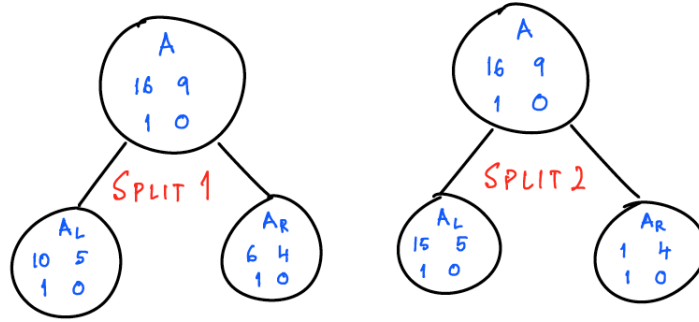


Figure 1.7: Choosing a split.

For Split 1 we have,

$$\begin{aligned} \text{Red}_{\text{Gini}}(\text{Split 1}) &= 2 \cdot \frac{9}{25} \cdot \frac{16}{25} - \frac{15}{25} \left(2 \cdot \frac{10}{15} \cdot \frac{5}{15} \right) - \frac{10}{25} \left(2 \cdot \frac{6}{10} \cdot \frac{4}{10} \right) \\ &= 0.0021 \end{aligned}$$

On the other hand, for Split 2,

$$\begin{aligned} \text{Red}_{\text{Gini}}(\text{Split 2}) &= 2 \cdot \frac{9}{25} \cdot \frac{16}{25} - \frac{20}{25} \left(2 \cdot \frac{15}{20} \cdot \frac{5}{20} \right) - \frac{5}{25} \left(2 \cdot \frac{1}{5} \cdot \frac{4}{5} \right) \\ &= 0.098 \end{aligned}$$

Since Split 2 has a higher reduction in impurity than Split 1; the CART algorithm prefers Split 2.